Name: Dinky Aswani	Class: T1	Roll No: 2401006
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

Experiment no. 03

Aim: To deploy multi-service application using Docker compose

LO4: To automate provisioning, configuration, container orchestration, and cloud resource management using Ansible, Docker, Kubernetes, and Terraform.

Theory:

Docker Compose is a tool provided by Docker that is used to define and run multi-container applications using a single YAML configuration file (docker-compose.yml). Instead of creating and starting each container separately using individual docker run commands, Docker Compose allows all the services required by an application — such as a web server, a database, and supporting tools — to be defined together in one file, and started, stopped, or managed together using a single command such as docker compose up. This directly supports the aim of this experiment: deploying a multi-service application as one unit rather than as separate, manually managed containers.

In real-world applications, a single container is rarely sufficient — a typical application may require a web server, a database, a caching layer, and other supporting services running together and communicating with each other. Managing each of these manually with individual docker run commands is tedious and error-prone.

Docker Compose solves this by allowing all services, along with their images, environment variables, ports, volumes, and dependencies, to be defined declaratively in the dockercompose.yml file. A single command, docker compose up, then reads this file and creates, configures, and starts all the defined services together, connected on a common Composemanaged network so they can communicate with each other using their service names as hostnames.

Key concepts used in this experiment:

Service: A single container definition within the compose file (e.g. web, database, phpmyadmin). Each service specifies the image to use, ports to expose, environment variables, and other configuration.

Image: The base image each service is built from — nginx for the web server, mysql:8.0 for the database, and phpmyadmin/phpmyadmin for the database admin UI.

Ports mapping ("host:container"): Maps a port on the host machine to a port inside the container, e.g. "8080:80" exposes the container's port 80 on the host's port 8080.

Environment variables: Used to configure a service at startup without modifying the image itself — here they set the MySQL root password, database name, and user credentials, and tell phpMyAdmin which database host to connect to.

Volumes: Named, persistent storage (mysql-data) that keeps database data intact even if the container is stopped, removed, or recreated.

depends_on: Specifies that one service depends on another, influencing the order in which containers are started (phpmyadmin is started after database).

This experiment uses three services instead of the minimum two, to demonstrate a more realistic multi-service setup: an nginx web server, a MySQL database, and a phpMyAdmin instance for

managing that database through a browser-based interface — showing how Compose handles inter-service communication (phpMyAdmin connecting to the database service purely by its service name, "database").

DOCKER COMPOSE FILE (docker-compose.yml)

The following configuration was created inside the compose-demo directory and saved as dockercompose.yml:

```
version: '3.8'
 services:    web:    image: nginx
container_name: webserver
ports:      - "8090:80"
   database:    image: mysql:8.0
container_name: mysqlserver
environment:
     MYSQL_ROOT_PASSWORD: root123
     MYSQL_DATABASE: studentdb
     MYSQL_USER: student
MYSQL_PASSWORD: student123    ports:
- "3307:3306"    volumes:
- mysql-data:/var/lib/mysql
   phpmyadmin:    image:
phpmyadmin/phpmyadmin
container_name: phpmyadmin
environment:      PMA_HOST:
database
     PMA_USER: student
PMA_PASSWORD: student123
ports:      - "8081:80"
depends_on:
- database

volumes:   mysql-data:
```

PROCEDURE

The following steps were carried out to set up and deploy the application:

#	Step / Command	Description
1	mkdir compose-demo	Creates a new project directory for this experiment.
2	cd compose-demo	Moves into the newly created directory.
3	(create docker-compose.yml)	The compose file is created using Notepad (saved as "All Files" so it is not saved with a .txt extension) inside the compose-demo folder.
4	dir	Lists the contents of the folder to confirm dockercompose.yml was saved correctly.
5	docker compose up -d	Reads docker-compose.yml and creates + starts all three services (webserver, mysqlserver, phpmyadmin) in detached (background) mode.

How the services connect

The web service (nginx) is accessible from the host browser at http://localhost:8090.

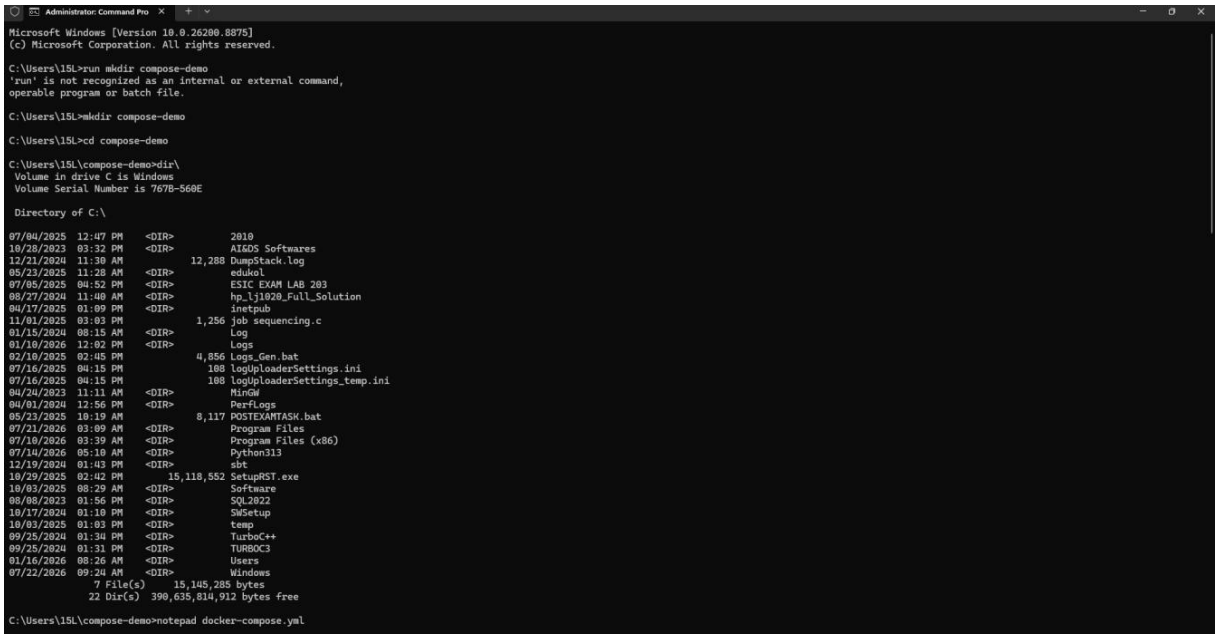
The database service (MySQL) is accessible on host port 3307, with credentials and database name set via environment variables, and its data persisted in the mysql-data volume.

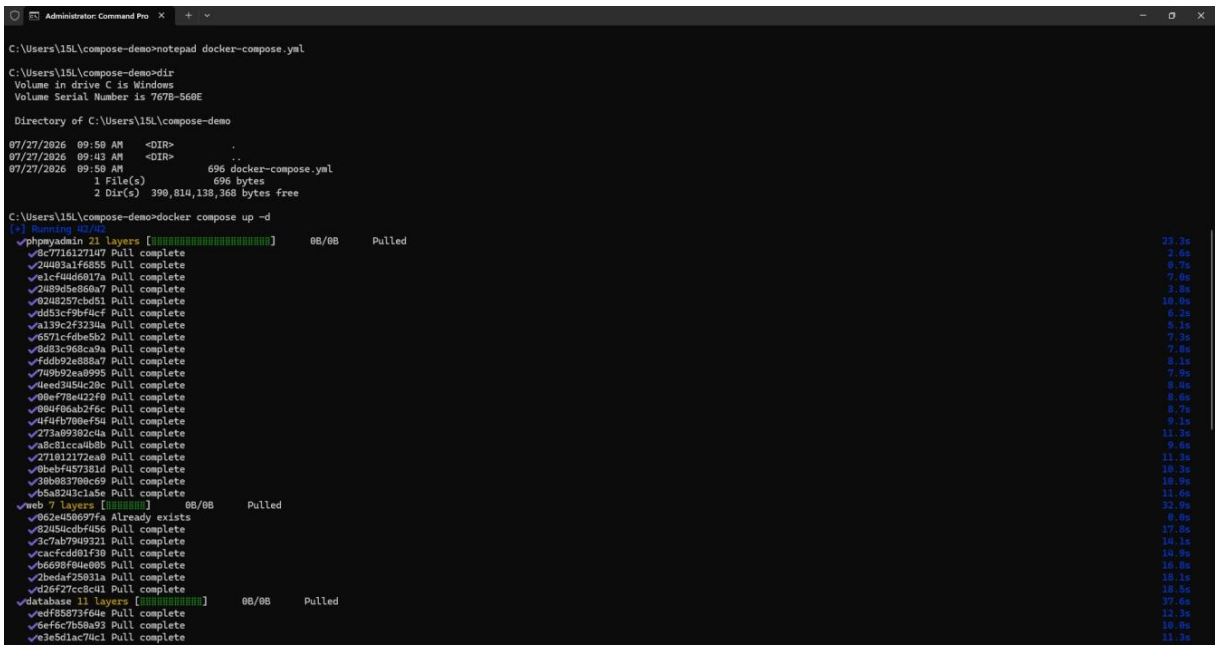
The phpmyadmin service is accessible at http://localhost:8081, and connects internally to the database service using the hostname "database" (its Compose service name) rather than an IP address — this works because Compose automatically places all services on the same internal network.

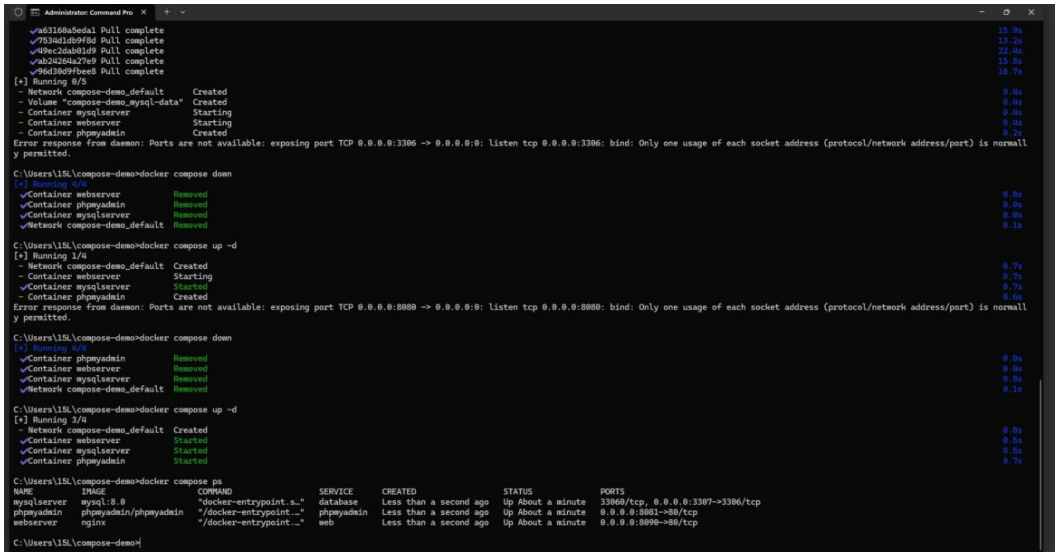
Note

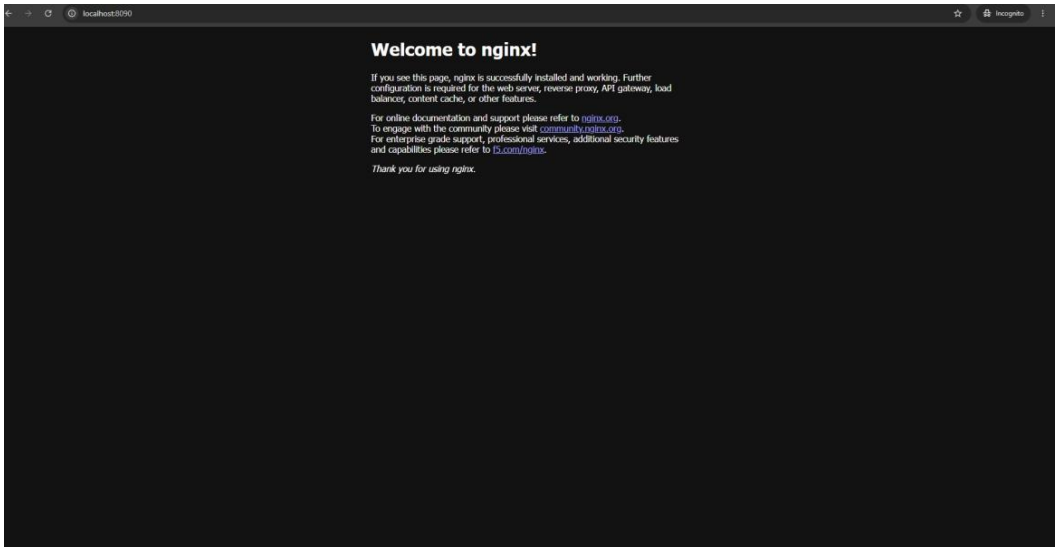
During deployment, docker compose up -d initially failed because the default host ports (3306 and 8080) specified in the file were already in use on the host machine. This was resolved by changing only the host-side port numbers to 3307 and 8090 respectively in docker-compose.yml, without altering the container-side ports, after which docker compose down followed by docker compose up -d started all three services successfully.

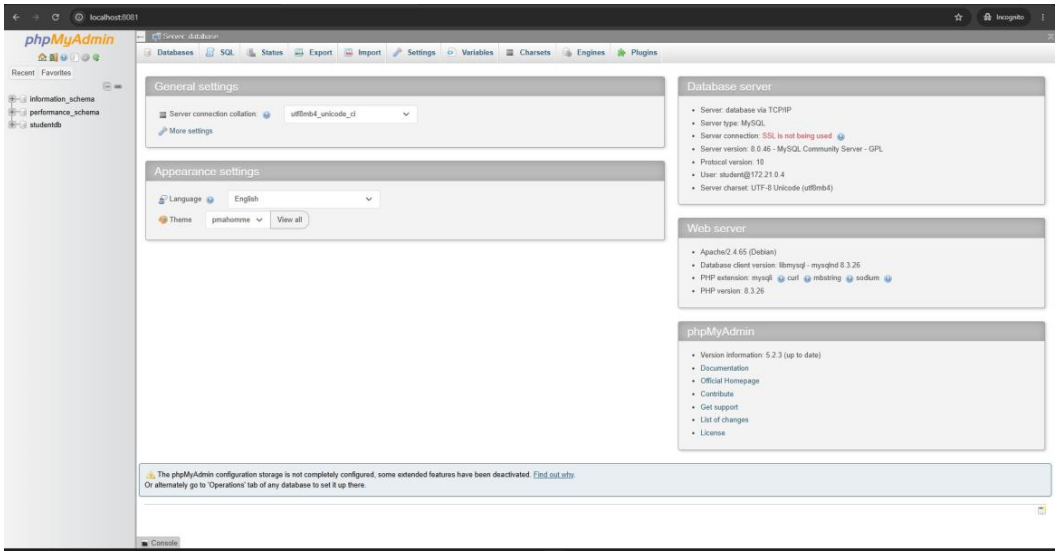
OBSERVATIONS / SCREENSHOTS


Administrator: Command Pro
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\Users\15L>run mkdir compose-demo
'run' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\15L>mkdir compose-demo
C:\Users\15L>cd compose-demo
C:\Users\15L\compose-demo>dir\
Volume in drive C is Windows
Volume Serial Number is 767B-560E
Directory of C:\
07/04/2025 12:47 PM <DIR> 2010
10/28/2023 03:32 PM <DIR> AI&DS Softwares
12/21/2024 11:30 AM 12,288 DumpStack.log
05/23/2025 11:28 AM <DIR> edukol
07/05/2025 04:52 PM <DIR> ESIC EXAM LAB 203
08/27/2024 11:40 AM <DIR> hp_lj1020_Full_Solution
04/17/2025 01:09 PM <DIR> inetpub
11/01/2025 03:03 PM 1,256 job sequencing.c
01/15/2024 08:15 AM <DIR> Log
01/10/2026 12:02 PM <DIR> Logs
02/10/2025 02:45 PM 4,856 Logs_Gen.bat
07/16/2025 04:15 PM 108 logUploaderSettings.ini
07/16/2025 04:15 PM 108 logUploaderSettings_temp.ini
04/24/2023 11:11 AM <DIR> MinGW
04/01/2024 12:56 PM <DIR> PerfLogs
05/23/2025 10:19 AM 8,117 POSTEXAMTASK.bat
07/21/2026 03:09 AM <DIR> Program Files
07/10/2026 03:39 AM <DIR> Program Files (x86)
07/14/2026 05:10 AM <DIR> Python313
12/19/2024 01:43 PM <DIR> sbt
10/29/2025 02:42 PM 15,118,552 SetupRST.exe
10/03/2025 08:29 AM <DIR> Software
08/08/2023 01:56 PM <DIR> SQL2022
10/17/2024 01:10 PM <DIR> SWSetup
10/03/2025 01:03 PM <DIR> temp
09/25/2024 01:34 PM <DIR> TurboC++
09/25/2024 01:31 PM <DIR> TURBOC3
01/16/2026 08:26 AM <DIR> Users
07/22/2026 09:24 AM <DIR> Windows
7 File(s) 15,145,285 bytes
22 Dir(s) 390,635,814,912 bytes free
C:\Users\15L\compose-demo>notepad docker-compose.yml


Administrator: Command Pro
C:\Users\15L\compose-demo>notepad docker-compose.yml
C:\Users\15L\compose-demo>dir
Volume in drive C is Windows
Volume Serial Number is 767B-560E
Directory of C:\Users\15L\compose-demo
07/27/2026 09:50 AM <DIR> .
07/27/2026 09:43 AM <DIR> ..
07/27/2026 09:50 AM 696 docker-compose.yml
1 File(s) 696 bytes
2 Dir(s) 390,814,138,368 bytes free
C:\Users\15L\compose-demo>docker compose up -d
[+] Running 42/42
phpmyadmin 21 layers [####################] 0B/0B Pulled 23.3s
8c7716127147 Pull complete 2.6s
2448f3a1f6055 Pull complete 8.7s
e1cf44d6017a Pull complete 7.0s
2489d5e860a7 Pull complete 3.8s
0248257cbd51 Pull complete 10.0s
dd53cf9bf4cf Pull complete 6.2s
a139c2f3234a Pull complete 5.1s
6571cfdbe5b2 Pull complete 7.3s
8d83c968ca9a Pull complete 7.8s
fddb92e888a7 Pull complete 8.1s
749b92ea0995 Pull complete 7.9s
4eed3454c20c Pull complete 8.4s
00ef78e422f0 Pull complete 8.6s
004f06ab2f6c Pull complete 8.7s
4f4fb700ef54 Pull complete 9.1s
273a80382c4a Pull complete 11.3s
a8c81cca4b8b Pull complete 9.6s
271012172ea0 Pull complete 11.3s
0bebf457381d Pull complete 10.3s
38b083700c69 Pull complete 10.9s
b5a8243c1a5e Pull complete 11.6s
web 7 layers [#######] 0B/0B Pulled 32.9s
062e4506974a Already exists 0.0s
82454cdbf456 Pull complete 17.8s
3c7ab7949321 Pull complete 14.1s
cacfcdd01f30 Pull complete 14.9s
b6698f04e005 Pull complete 16.8s
2bedaf25031a Pull complete 18.1s
d26f27cc8c41 Pull complete 18.5s
database 13 layers [##########] 0B/0B Pulled 37.6s
edf85873f64e Pull complete 12.3s
6ef6c7b50a93 Pull complete 10.0s
e3e5d1ac74c1 Pull complete 11.3s







DOCKER COMPOSE COMMANDS USED

The following Docker Compose commands were used to manage the lifecycle of the multiservice application:

#	Command	Description
1	docker compose up -d	Starts all services defined in the compose file, in detached mode.
2	docker compose ps	Shows the status of all running services defined in the compose file.
3	docker compose logs	Displays combined logs from all services, useful for debugging.
4	docker compose images	Lists the images used by each service in the compose file.
5	docker compose config	Validates the syntax of the compose file and displays the fully resolved configuration.
6	docker compose stop	Stops all running services without removing the containers.
7	docker compose start	Starts services that were previously stopped.
8	docker compose restart	Restarts all containers defined in the compose file.
9	docker compose down	Stops and removes all containers and the default network, but keeps named volumes.
10	docker compose down -v	Stops and removes all containers, networks, AND named volumes (deletes database data) — run this last.

CONCLUSION

Through this experiment, a multi-service application consisting of a web server, a database, and a database administration tool was successfully deployed using Docker Compose. Instead of creating and managing each container individually, a single docker-compose.yml file was used to declaratively define all three services, their images, ports, environment variables, and persistent storage. The docker compose up -d command started all services together with a single instruction, and docker compose ps confirmed that all three containers were running and correctly networked. This experiment demonstrates the core advantage of Docker Compose over standalone Docker commands: it simplifies the deployment and orchestration of applications composed of multiple interdependent services, allows services to communicate using simple service names instead of IP addresses, and makes the entire application's configuration reproducible and versioncontrollable through a single file.